

程设 1 小班辅导

无 12 李羿璇

目录

1 引言	1
1.1 程设 1 我们应当掌握什么	1
1.2 背景知识	1
2 知识点梳理	2
2.1 基本语法	2
2.1.1 程序执行顺序	2
2.1.2 数组与指针	4
2.1.3 函数调用	5
2.1.4 综合运用	6
2.2 输入输出	8
2.2.1 标准输入输出	8
2.2.2 文件读写	8
2.2.3 通过命令行的传参	9
2.3 字符串相关问题	9
2.3.1 字符串的简单使用	9
2.3.2 字符串比较问题	10
2.3.3 字符串长度问题	11
2.4 杂项	11
2.4.1 define 问题	11
2.4.2 static 问题	11
2.4.3 结构体问题	12
3 扩展	13
3.1 C 语言程序的预处理、编译、汇编、链接	13
4 补充 & 答疑 & 反馈	14

1 引言

1.1 程设 1 我们应当掌握什么

- 初步了解什么是编程，掌握程序设计的基本方法与思想
- 会使用 C 语言编写简单程序

1.2 背景知识

C 语言是由著名的 Brian Kernighan 和 Dennis Ritchie 发明的语言，继承于 B 语言。在 1978 年，两位科学家正式发布了第一版 C 语言指导书 **The C Programming Language**，介绍了 C 语言的语法。这一版 C 语言常被称作“K&R C”或“C78”。K&R C 与我们现在的 C 语言差别还很大。

1983 年，美国标准化组织 ANSI 成立 C 语言委员会，对 C 语言进行标准化，并于 1989 年发布了影响最为深远的第一个标准化的 C 语言版本，常被称作 ANSI C 或 C89；1990 年，国际

标准委员会 ISO 对 C 语言进行了标准化，并发布了 C90（与 C89 几乎完全相同）。现在，C 语言由 ISO 的 WG14 工作组负责标准化工作，C 语言历代标准如下：

发布年份	称呼	备注
史前	K&R C	最早版本
1989/1990	C89/C90	第一个 C 标准，影响广泛
1999	C99	增加从 C++ 偷来的新语法
2011	C11	增加没什么大用的新语法
2018	C17/C18	小补丁
2024	C23	本来计划 2023 发布结果鸽了

表 1 C 语言历代标准

问：那咱们的程设是哪一个标准呢？

答：咱们的程设是微软（Microsoft）发布的 VS2008“标准”！（bushi）

当然最接近的标准是 C89

2 知识点梳理

2.1 基本语法

2.1.1 程序执行顺序

主要涉及知识点：基本的表达式、逗号表达式、分支、循环

可能的一些考法：

2022 年 13 题

```
#include <stdio.h>
int main()
{
    int a = 2, i;
    for (i = 1; i <= 8; i++)
    {
        if (a >= 6)
            break;
        if (a % 2)
        {
            a += 4;
            continue;
        }
        a = a - 5;
    }
    printf("%d,%d", i, a);
}
```

答案：5,9

备注：最常见的题型，需要跟着题目一步一步进行操作，主要考察答题时的毅力。

表达式的真假问题：只有 0 会被认为是假，其他都会被认为真。例如，if(-1) 会被认为是 if(true)。

2019 年 6 题

```
#include <stdio.h>
int f(int a, int b)
{
    int c;
    c = a + b;
    return c;
}
int main()
{
    int x = 7, y = 8, z = 9;
    printf("%d\n", f((x++, y--, x + y), z--));
    return 0;
}
```

答案: 24

备注: 考察逗号表达式的执行先后顺序, 需要知道逗号表达式从左到右依次执行 (且存在序列点), 以最后一个子表达式的值作为整个逗号表达式的值。

2019 年 7 题

```
#include <stdio.h>
int main()
{
    int k;
    for (k = 1; k < 5;)
    {
        switch (k)
        {
            case 1:
                printf("%d ", k++);
            case 2:
                printf("%d ", k++);
                break;
            case 4:
                printf("%d ", k++);
            case 3:
                printf("%d ", k++);
                break;
            default:
                printf("full\n");
        }
    }
    return 0;
}
```

答案: 1 2 3 4 5

备注: 主要考察对 switch 表达式的理解, 需要注意 case 后不加 break 的话就不会跳回去, 会继续执行。

补充: switch 的设计灵感来源于汇编语言的跳转表, 是跳转表的高级语言中的“抽象”。不过不必用跳转表实现, 完全取决于具体实现。

2.1.2 数组与指针

指针可以理解为变量的“门牌号”，变量被存储在计算机的内存中，指针则告诉了我们某个变量具体存储在了具体哪个内存地址中，通过指针可以找到对应的内存位置。

```
#include <stdio.h>
int main()
{
    int a[2] = {1, 2};
    int *p1 = a, *p2 = &a[1];
    printf("%p,%p\n", p1, p2);
}
```

输出结果：0x7ffc7b0d1c78,0x7ffc7b0d1c7c，可以看到差了4（其实就是int的4个字节）。

此外，这部分的试题还可能涉及 malloc 等动态分配内存的方式。

下面是一些往年试题：

2019 年 1 题

```
#include <stdio.h>
int main()
{
    int a[3][3] = {1, 3, 5, 7, 9, 11, 13, 15, 19}, i, j, sum1 = 0, sum2 = 0;
    for (i = 0; i < 3; i++)
        for (j = 0; j < 3; j++)
            if (i == j)
                sum1 += a[i][j];
    for (i = 0; i < 3; i++)
        for (j = 2; j >= 0; j--)
            if (i + j == 2)
                sum2 += a[i][j];
    printf("%d %d\n", sum1, sum2);
    return 0;
}
```

答案：29 27

备注：对数组的简单操作，考察耐心。需要注意，初始化和赋值是两个不同的操作！

例如，int a[10] = {1, 2, 3, ...}; 是正确的

int a[10]; a[10] = {1, 2, 3, ...}; 是错误的。

2019 年 14 题

```
#include <stdio.h>
int main()
{
    int a[3][2] = {1, 2, 3, 4, 5, 6};
    int *p[3], i, j;
    for (i = 0; i < 3; i++)
        p[i] = a[i];
    for (i = 1; i < 3; i++)
        for (j = 0; j < 2; j++)
            printf("%d ", (*(p + i) + j));
}
```

```

    return 0;
}

```

答案：3 4 5 6

备注：这里考察了数组到指针的退化，二维数组会向指向数组的指针（行指针）退化。一般地，类型 T 的数组向指向 T 的指针退化。此处 T 为 $\text{int}[]$ 。

拓展：一般来说， k 维数组可以看作 $k-1$ 维数组的数组。因此，一般地，一个 k 维数组会向指向 $k-1$ 维数组的指针退化。

2019 年 5 题

```

#include <stdio.h>
int main()
{
    int **k, a[6] = {3, 15, 17, 19, 11, 5}, z;
    int* p = a;
    k = &p;
    z = *p;
    p = p + 1;
    z = z + **k;
    printf("%d\n", z);
    return 0;
}

```

答案：18

备注：考察指针与数组之间的转换。对指针+1 相当于是指向下一个元素，但是指针指向的地址不一定是直接+1，取决于具体指向的数据类型。由此可见，指针的数值不能简单地看作一个数。

```

#include <stdio.h>
int main()
{
    int a[2] = {1, 2};
    int *p = a, *q = &a[1];
    printf("%d\n", q - p); // 1
}

```

2.1.3 函数调用

2022 年 9 题

```

#include <stdio.h>
void swap(int** a, int** b)
{
    int* t;
    t = *a;
    *a = *b;
    *b = t;
}
int main()
{
    int i = 4, j = 8, *p = &i, *q = &j;
    swap(&p, &q);
    printf("%d,%d,%d,%d\n", i, j, *p, *q);
}

```

答案：4,8,8,4

备注：函数传递的是值，例如，下面的操作不会改变 a,b 的值。由此可见，如果希望在函数里修改某种变量的数值来在函数外有体现，需要将对应的指针传进去。

```
void swap(int a, int b)
{
    int temp = a;
    a = b;
    b = temp;
}
```

2022 年 11 题

```
#include <stdio.h>
int f1(int n)
{
    return n + 2;
}
int f2(int n)
{
    return n * n + 3;
}
int f3(int n)
{
    return n * n * n + 5;
}
int func(int (*f)(int), int i)
{
    return f(i);
}
int main()
{
    int s = 0, i = 1;
    s += func(f1, ++i);
    printf("%d,", s);
    s += func(f2, ++i);
    printf("%d,", s);
    s += func(f3, ++i);
    printf("%d\n", s);
}
```

答案：4,16,85

备注：考察了函数指针的使用，函数指针是 C 语言中多态的一种体现。题目本身并不难，小心不要算错即可。

2.1.4 综合运用

2019 年 16 题

```
#include <stdio.h>
#include <malloc.h>
void f(int* p, int (*a)[3], int n)
{
    int i, j;
```

```

    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
        {
            *p = a[i][j] + 1;
            p++;
        }
}
int main()
{
    int *p, a[3][3] = {{1, 3, 5}, {2, 4, 6}, {7, 8, 9}};
    p = (int*)malloc(sizeof(int) * 100);
    f(p, a + 1, sizeof(a) / sizeof(a[0]));
    printf("%d %d\n", p[2], p[5]);
    return 0;
}

```

答案: 7 10

备注: 本题涉及考点较多。包括但不限于

- malloc 的使用: malloc 返回一个 void*, 需要通过强转成对应的类型
- 数组指针与指针数组的区别:
 - 数组指针: int (*p)[3]; 指向一个长度为 3 的 int 数组的指针, 有时也称之为行指针
 - 指针数组: int *p[3]; 存放了 3 个指针的数组
- sizeof 运算符的使用: 数组的 sizeof() 会获得数组的长度×数组的数据类型长度, 例如上题中 sizeof(a)=36、sizeof(a[0])=12。
- 指针的加减: a+1 指向二维数组 a 的第二行
- 函数的传参

此外, 本题存在未定义行为, 上述讲解均只适用于考试。

2019 年 18 题

```

#include <stdio.h>
int f(int (*p)[4], int n)
{
    int i;
    int m;
    m = **p;
    for (i = 1; i < n; i++)
        if (*(*p + i) > m)
            m = *(*p + i);
    return m;
}
int main()
{
    int a[4][3] = {1, 2, 3, 4, 5, 6, 7, 6, 5, 4, 3, 2}, n = 3 * 4;
    printf("%d\n", f(a, n));
    return 0;
}

```

答案: 7

备注：题目本身不难，主要涉及 `int (*p)[3]` 向 `int (*p)[4]` 的转换。C 语言是允许这样的隐式转换的，但是换到下学期大家学习的 C++ 语言里则会报错，最好还是别这么写×

2.2 输入输出

2.2.1 标准输入输出

主要考点：scanf、printf、gets、puts、getchar、putchar 等

2019 年 3 题：输入为 HOW DO YOU DO 回车

```
#include <stdio.h>
int main()
{
    char str1[] = "how do you do";
    char str2[20], str3[20], *p1, *p2, *p3;
    p1 = str1;
    p2 = str2;
    p3 = str3;
    scanf("%s", p2);
    gets(p3);
    printf("%s ", p2);
    printf("%s ", p1);
    printf("%s\n", p3);
    return 0;
}
```

答案：HOW how do you do （两个空格）DO YOU DO

备注：注意 scanf 读到回车/空格为止，且不会读入回车/空格，gets 读到回车为止，且不会读入回车，但会读入空格。

对于本题而言，p3 读入了 HOW 与 DO 之间的空格。

2.2.2 文件读写

2022 年 7 题

```
#include <stdio.h>
int main()
{
    FILE* fp;
    int k, n, a[6] = {11, 32, 43, 54, 65, 76};
    fp = fopen("d2.dat", "w");
    fprintf(fp, "%d%d\n%d\n", a[0], a[1], a[2]);
    fprintf(fp, "%d%d%d\n", a[3], a[4], a[5]);
    fclose(fp);
    fp = fopen("d2.dat", "r");
    fscanf(fp, "%d%d", &k, &n);
    printf("%d,%d\n", k, n);
    fclose(fp);
}
```

答案：1132,43

备注：注意二进制读写与这样以字符串形式读写的区别。二进制读写是可以区分上面先后写入的 `a[0]`, `a[1]` 的，而字符串则很难区分。

改编自 2018 年期末某题

```
#include <stdio.h>
int main(void)
{
    FILE* fp;
    char a[10] = {"tsinghua"}, b[10], *p;
    p = b;
    fp = fopen("d.dat", "w+");
    fwrite(a, sizeof(char), 6, fp);
    fseek(fp, sizeof(char) * 3, SEEK_SET);
    fgets(p, 5, fp);
    fclose(fp);
    printf("%c, %c", *(p + 2), b[2]);
    return 0;
}
```

答案: h, h

备注: 此处 fseek 的用法应该注意。

```
int fseek(FILE *stream, long int offset, int whence)
```

其中 offset 是偏移量, whence 一般在 SEEK_SET (开头)、SEEK_CUR (当前)、SEEK_END (结尾) 中选一个。最终文件指针会跳转到 fseek 的目标位置。

此外, fgets 会读到换行或文件尾或最大字符数, 且会在字符串尾补上 \0。本题中从 3 位置 (n) 开始读, 读入了 "ngh\0" 四个字符。

2.2.3 通过命令行的传参

2022 年 16 题: 生成 test.exe 后, 通过 test THIS IS OUR FINAL EXAM 调用

```
#include <stdio.h>
int main(int argc, char* argv[])
{
    int k;
    printf("%d,", argc);
    for (k = 1; k < argc; k++)
    {
        if (k % 2)
            printf("%s,", argv[k]);
        else
            printf("%c,", *(argv[k] + 1));
    }
}
```

答案: 6,THIS,S,OUR,I,EXAM,

备注: 通过命令行传参后, argc 是传入的参数个数 (上面为 6 个, 包括调用程序所用的 test 也包含其中), argv 的每一行都是一个字符串, 指向具体的参数。

2.3 字符串相关问题

2.3.1 字符串的简单使用

2022 年 20 题

```

#include <stdio.h>
int main()
{
    char a[6][10] = {"China", "Beijing", "Winter", "Olympics", "Welcome", "You"};
    char* p[6];
    int i;
    for (i = 0; i < 6; i++)
        p[i] = a[i];
    printf("%s", p[1]);
    for (i = 5; i >= 0; i -= 2)
    {
        printf("%c", *p[i]);
        printf("%s", p[i]);
    }
}

```

答案: BeijingYYouOOlympicsBBeijing

备注: 题目本身没有难度, 注意字符串的基本概念, 考试时注意细节

2.3.2 字符串比较问题

2022 年 3 题

```

#include <stdio.h>
#include <string.h>
void fun(char* s[], int n)
{
    char* t;
    int i, j;
    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (strcmp(s[i], s[j]) > 0)
            {
                t = s[i];
                s[i] = s[j];
                s[j] = t;
            }
        }
    }
}

int main()
{
    char* ss[] = {"huang", "song", "yang", "CCCCAA", "bbccDD"};
    fun(ss + 1, 3);
    printf("%s,%s\n", ss[0], ss[2]);
}

```

答案: huang,song

备注: strcmp 按照 ASCII 表的顺序排序, 注意小写字母永远比大写字母要大。

此外, 简单的排序算法 (本题是冒泡排序) 大家应当掌握。

2.3.3 字符串长度问题

2019 年 17 题

```
#include <stdio.h>
#include <string.h>
int main()
{
    char st[21] = "hahaa\0\t'\\";
    printf("%d,%d,%s\n", strlen(st), sizeof(st), st);
    return 0;
}
```

答案：5,21,hahaa

字符串中的 `\0` 影响 `strlen`，但不影响数组长度（不影响 `sizeof`），影响输出

2.4 杂项

2.4.1 define 问题

2019 年 10 题

```
#include <stdio.h>
#define A 4
#define B(x) A* x / 2
int main()
{
    int a = 5, b = 6, c;
    c = B(a + b);
    printf("%d\n", c);
    return 0;
}
```

答案：23

备注：define 只是做了简单的替换，因此不能先算出 `a+b` 再代入计算。

事实上，include 做的事情也差不多（见扩展部分）。

2.4.2 static 问题

2019 年 9 题

```
#include <stdio.h>
int f(int a)
{
    static int c = 2;
    int b = 5;
    b = b + 1;
    c = c + 1;
    return a + b + c;
}
int main()
{
    int i;
    static int a = 2;
    for (i = 0; i < 3; i++)
```

```

        printf("%d ", f(a++));
    return 0;
}

```

答案：11 13 15

备注：考察静态存储期。注意（在语法层面上）只在第一次执行到该处时进行初始化。

2.4.3 结构体问题

2019 年 11 题

```

#include <stdio.h>
struct HAP
{
    int x;
    int y;
    struct HAP* p;
} h[2];
int main()
{
    h[0].x = 3;
    h[0].y = 3;
    h[1].x = 4;
    h[1].y = 4;
    h[0].p = &h[1];
    h[1].p = h;
    printf("%d%d\n", h[0].p->x, h[1].p->y);
    return 0;
}

```

答案：43

备注：题目本质上是 h[0] 和 h[1] 之间互相指。此外，关于结构体的对齐，有一些知识点：

默认对齐规则：

- 对齐位数为最大的 scalar 类型的大小，但不超过机器字长（scalar：算术类型、枚举类型、指针类型（注：不包括数组、结构体））
- 结构体的大小是对齐位数的正整数倍
- 每个 scalar 成员的地址相对于结构体起始位置的偏移量是 min{其大小, 机器字长} 的整数倍

```

• struct {
    char c;
    int32_t x; // int32_t 指四字节大小的整数
    int32_t y;
};
// 32-bit / 64-bit: 12bytes

```

```

• struct {
    int32_t x;
    int32_t y[2];
    char c[3];
};
// 32-bit / 64-bit: 16bytes

```

```

• struct {
    int32_t x;

```

```
double d;
};
// 32-bit: 12bytes
// 64-bit: 16bytes
```

3 扩展

3.1 C 语言程序的预处理、编译、汇编、链接

C/C++ 程序生成一个可执行文件的过程可以分为 4 个步骤：预处理（Preprocessing）、编译（Compiling）、汇编（Assembly）和链接（Linking）。

编译工具

针对不同平台，各大厂家设计了不同的 C++ 编译工具：

- MSVC：是微软开发的 C++ 编译器，主要用于 Windows，Visual Studio 中内置了 MSVC
- GCC：是 GNU 开发的编译器，支持 C、C++、Fortran、Go 等多种语言
- 其他：Clang、NVCC...此处不再赘述

在 Linux 上，可以通过

```
sudo apt-get install gcc g++
```

来安装 GCC 编译器。

预处理

假设我们有如下代码：

```
// hello.cpp
#include <iostream>

void hello()
{
    std::cout << "Hello, World!" << std::endl;
}

// main.cpp
void hello();

int main()
{
    hello();
    return 0;
}
```

预处理阶段，C++ 执行宏的替换、头文件的插入、删除条件编译中不满足条件的部分

```
g++ -E hello.cpp -o hello.i
```

编译

C++ 程序在编译阶段会将 C++ 文件转换为汇编文件。

```
g++ -S hello.i -o hello.s
g++ -S hello.cpp -o hello1.s # 可以直接从 .cpp 生成
```

汇编

汇编文件经过汇编生成目标文件（机器码），每一个源文件对应一个**目标文件**。

```
g++ -c hello.s -o hello.o
```

```
g++ -c main.cpp -o main.o # 可以直接从 .cpp 生成
```

二进制文件不能直接打开

链接

将每个源文件的目标文件链接起来，形成一个**可执行程序文件**

```
g++ hello.o main.o -o main
```

```
g++ hello.cpp main.cpp -o main # 可以直接从 .cpp 生成
```

一般地，我们约定 `.i` 为预处理后的文件，`.s` 为汇编文件，`.o` 为目标文件

4 补充 & 答疑 & 反馈

考试期间可能会遇到一些问题：

- 代码排版非常混乱，可读性极差，需要大家耐心读代码，切忌浮躁
- 手写空格：只要表示清楚意思即可，基本无须担心因为这个的格式扣分

大家提出的一些问题

- 指针的具体意义是什么？

指针可以理解为变量的“门牌号”，它的值是指向的变量存储在计算机中的地址。

- 有些很抽象的概念（编译、命令行）这些会考很多吗？以及补码啊 double 数存储啊这些记不住的东西会要求我算吗？

编译往年是不会考的，当然今年如果提供了往年题的话可能情况有变，难度也许会增加；命令行肯定会考，但是一般考的内容很简单，类似上面讲的内容。补码、double 数存储在程设中不会考，但是补码会在《数字逻辑与处理器基础》课程中涉及、double 数存储会在《数据与算法》中涉及，故最好还是有所了解。

- 指向指针的指针到底怎么指...指针数组和数组指针怎么用，链表怎么一个一个勾起来翻过来

指向指针的指针和正常指针实际上没有本质区别，因为指针本身也是一个变量，也要被存储在计算机的内存中，指向指针的指针实际上就是指向了这个指针变量。数组指针：`int (*p)[3]`；指向一个长度为 3 的 `int` 数组的指针，有时也称之为行指针；指针数组：`int *p[3]`；存放了 3 个指针的数组

链表的翻转：为什么不问问神奇的 chatgpt 呢（×）

在 C 语言中，可以使用三个指针来翻转单链表。以下是详细的步骤：

1. 初始化三个指针：`prev`、`current` 和 `next`。`prev` 指向 `NULL`，`current` 和 `next` 指向链表的头节点。
2. 在链表未结束前，执行以下操作：
 - 先将 `next` 指向 `current` 的下一个节点。
 - 然后将 `current` 的 `next` 指向 `prev`。
 - 最后，将 `prev` 和 `current` 向前移动一步。
3. 当链表遍历完后，`prev` 将指向新的头节点。

以下是对应的 C 代码：

```

struct Node {
    int data;
    struct Node* next;
};

struct Node* reverse(struct Node* head) {
    struct Node* prev = NULL;
    struct Node* current = head;
    struct Node* next = NULL;
    while (current != NULL) {
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }
    head = prev;
    return head;
}

```

在这段代码中，Node 是链表节点的结构体，它有一个 data 成员用于存储数据，一个 next 指针用于指向下一个节点。reverse 函数接收一个链表的头节点，返回翻转后的链表的头节点

- 指针的应用？指针的用途？

指针的核心意义在于可以让程序员能够自己手动管理内存，在增加大家写代码难度（bushi）的同时，也为编程提供了更多的灵活性。这里可以举个简单的例子：假设我们在开发一款游戏

```

struct State
{
    int a[10000];
    int b[10000];
};

```

现在我们需要一个 buffer 来接收服务器发来的消息、以及一个 current 来给用户使用。每一秒，我们都需要将 current 换成 buffer，然后接收新的 buffer。怎么做？

或许你会想到，用一个临时变量来做操作、引入一个标记 flag 来表示.....但无论如何，要么程序不会很优雅，要么就必然会引入很大的复杂度（别忘了这是一个非常大的结构体）。实际上，利用指针，就有很优雅的方法：

```

struct State state[2];
struct State *current = &state[0], *buffer = &state[1];

// 交换状态
struct State* temp = current;
current = buffer;
buffer = temp;

```

在其他更高级的语言，如 Python、Java 中，由于安全性等问题，裸指针基本不再使用。但这些语言的引用类型，其实和指针也比较类似，此处不再赘述。

- 常用字符串函数

事实上程设用到的真的不多，无非就是 strcat、strlen、strcpy、strcmp 这么几种（）其他的在程设上基本就不涉及了。

- 机考相关问题，怎么快速 Debug、怎么断点调试 ·

关于快速 Debug：重点在于知道自己每一行代码在干什么，希望达成怎么样的效果，才可以 Debug，不要凭想象 Debug；

关于断点调试：与刚才提到的“快速 Debug”结合，主要目的在于确定变量的值、确定函数的执行顺序，是否按照自己的预期进入了某个分支？ ...

反馈二维码（求个五星 QAQ）：



图 1 反馈二维码